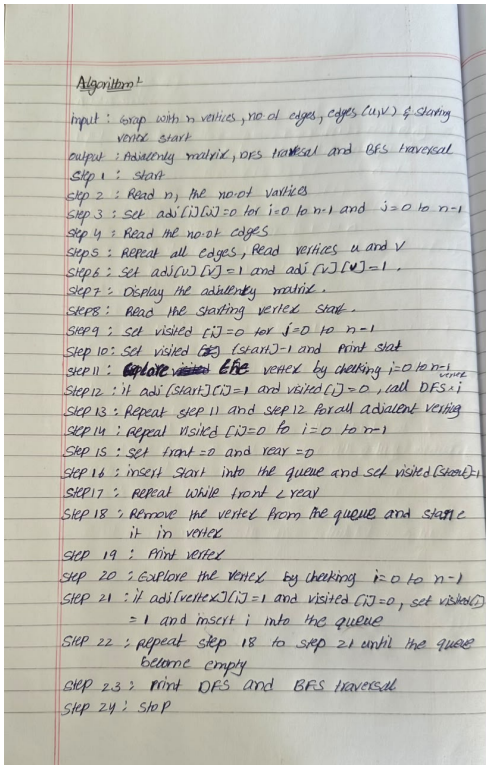
PRACTICAL - 8

AIM : Implementation of Graph and Searching (DFS and BFS).

ALGORITHM :



Algorithm:

Input : Graph with n vertices, no of edges, edges (u,v) & starting vertex start
Output : Adjacency matrix, DFS traversal and BFS traversal
Step 1 : Start
Step 2 : Read n, the no of vertices
Step 3 : Set adj[i][j]=0 for i=0 to n-1 and j=0 to n-1
Step 4 : Read the no of edges
Step 5 : Repeat all edges, Read vertices u and v
Step 6 : Set adj[u][v]=1 and adj[v][u]=1.
Step 7 : Display the adjacency matrix.
Step 8 : Read the starting vertex start.
Step 9 : Set visited[i]=0 for i=0 to n-1
Step 10 : Set visited[start]=1 and Print start
Step 11 : Explore the vertex by checking i=0 to n-1
Step 12 : if adj[start][i]=1 and visited[i]=0, call DFS(i)
Step 13 : Repeat step 11 and step 12 for all adjacent vertex
Step 14 : Repeat visited[i]=0 to i=0 to n-1
Step 15 : Set front=0 and rear=0
Step 16 : insert start into the queue and set visited[start]=1
Step 17 : Repeat while front < rear
Step 18 : Remove the vertex from the queue and store it in vertex
Step 19 : Print vertex
Step 20 : Explore the vertex by checking i=0 to n-1
Step 21 : if adj[vertex][i]=1 and visited[i]=0, set visited[i]=1 and insert i into the queue
Step 22 : Repeat step 18 to step 21 until the queue become empty
Step 23 : Print DFS and BFS traversal
Step 24 : Stop

PROGRAM :

```
#include <iostream>
using namespace std;
class Graph
{
public:
int adj[10][10];
int visited[10];
int n;
void getInput()
{
int edges;
cout << "Enter number of vertices: ";
cin >> n;
for(int i = 0; i < n; i++)
{
for(int j = 0; j < n; j++)
```

```
{
adj[i][j] = 0;
}
}
cout << "Enter number of edges: ";
cin >> edges;
cout << "Enter edges:\n";
for(int i = 0; i < edges; i++)
{
int u, v;
cin >> u >> v;
adj[u][v] = 1;
adj[v][u] = 1;
}
}
void displayGraph() {
cout << "\nAdjacency Matrix:\n";
for(int i = 0; i < n; i++){
for(int j = 0; j < n; j++)
{
cout << adj[i][j] << " ";
}
cout << endl;
}
}
void DFS(int start)
{
visited[start] = 1;
cout << start << " ";
for(int i = 0; i < n; i++){
if(adj[start][i] == 1 && visited[i] == 0) {
DFS(i);
}
}
}
void BFS(int start)
{
int queue[10];
int front = 0;
int rear = 0;
for(int i = 0; i < n; i++) {
visited[i] = 0;
}
```

```
visited[start] = 1;
queue[rear] = start;
rear++;
while(front < rear)
{
int vertex = queue[front];
front++;
cout << vertex << " ";
for(int i = 0; i < n; i++)
{
if(adj[vertex][i] == 1 && visited[i] == 0)
{
visited[i] = 1;
queue[rear] = i;
rear++; }
}
}
}
};
int main()
{
Graph g;
int start;
g.getInput();
g.displayGraph();
cout << "\nEnter starting vertex: ";
cin >> start;
for(int i = 0; i < g.n; i++)
{
g.visited[i] = 0;
}
cout << "\nDFS Traversal: ";
g.DFS(start);
cout << "\nBFS Traversal: ";
g.BFS(start);
cout << endl;
return 0;
}
```

OUTPUT :

```
Output

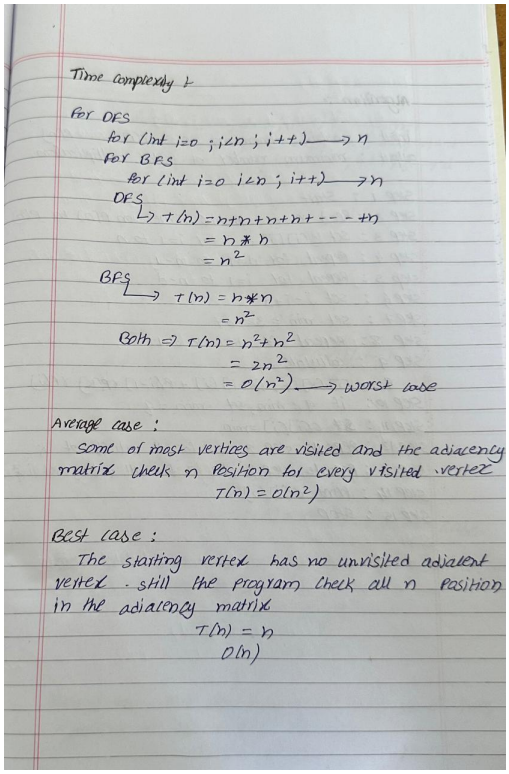
Enter number of vertices: 5
Enter number of edges: 5
Enter edges:
0
1
0
2
1
3
1
4
2
5

Adjacency Matrix:
0 1 1 0 0
1 0 0 1 1
1 0 0 0 0
0 1 0 0 0
0 1 0 0 0

Enter starting vertex: 0

DFS Traversal: 0 1 3 4 2
BFS Traversal: 0 1 2 3 4
```

TIME COMPLEXITY :



CONCLUSION :

I implemented a Graph data structure and performed graph traversal using Depth First Search (DFS) and Breadth First Search (BFS). I represented the graph using an adjacency list and used a visited array to keep track of the vertices that had already been explored. In DFS, I explored each adjacent vertex deeply before backtracking, while in BFS, I used a queue to visit the vertices level by level. I tested both algorithms with different graph inputs and verified their traversal orders. Through this implementation, I understood the practical difference between DFS and BFS and learned how both algorithms can efficiently traverse a graph with a time complexity of O(V + E).